DLDCA Inlab: Can You Hear The Music?

Week 9

October 16, 2026

"ChampSim is like sheet music. The important thing isn't can you read the C++, it's can you hear it. Can you hear the music, Robert?"

— *Oppenheimer*, if Bohr gave him a cache simulator probably

Welcome to Simulator World!

We will learn how memory hierarchy works in modern processors by running ChampSim, a widely used open-source simulator. **This is a larger codebase considering the lab work you guys have done so far, so it might look daunting; but we are going to break down the concepts one by one, and map them to parts of the codebase and also cover general workings of the codebase at large.**

ChampSim does not run a program directly. It replays something called a *trace* which is a recording of the instructions of a real program execution, together with the addresses they read and wrote.

ChampSim is a trace-based, cycle-level simulator, meaning, it literally encodes the logic of a modern out of order processor, in C++ code, where the input parameters are the configuration options (for cpu, cache and memory parameters) + the trace file. Please keep this image in mind : every cache operation and signal/data transfer is like calling a function or updating a struct somewhere..

E.g. if you do not vary traces and the cache parameters, ChampSim will run the same program with the same cache parameters on the same trace and output the same results. To the dot.

This lab looks at the cache hierarchy through ChampSim: how a request travels from the core through the L1, L2 and last-level cache (LLC) to memory, how caches handle multiple misses concurrently, and how the replacement policy picks the line to evict. You then add a counter to the L2C, write two replacement policies for the L2C, and check them against our reference results.

- **Task 1**: count read hits per way in ChampSim's L2C and print them with the other statistics; counting misses per set is optional.
- **Task 2**: write LRU yourself and show that it gives exactly the same result as ChampSim's own `lru`.
- **Task 3**: write a replacement policy that ranks lines by the instruction that brought them in.

Necessary tools

We use `g++`, `make` and `python3` to build ChampSim, and `xz` to read the traces. Check that they are installed:

```
$ g++ --version
g++ ([linux-flavour] [version]) [version]
$ make --version
GNU Make [version]
$ python3 --version
Python [version]
$ xz --version
xz (XZ Utils) [version]
```

On the lab machines the traces are in `/opt/CS231_Traces`. We use four programs from the SPEC CPU benchmark suites: `soplex` (a linear-program solver), `bwaves` (a fluid simulation that streams through large arrays), `mcf` (vehicle scheduling, with irregular pointer-chasing reads) and `gcc` (the C compiler).

```
$ ls /opt/CS231_Traces
602.gcc_s-2226B.champsimtrace.xz   605.mcf_s-994B.champsimtrace.xz
603.bwaves_s-891B.champsimtrace.xz  soplex_66B.trace.xz
```

The lab `makefile` finds a trace by its name: `trace=soplex` picks the file matching `*soplex*.xz`. To use traces in another folder, run `export CS231_TRACE_DIR=[that folder]`.

At home, download the same four files (names as listed above) from the DPC-3 trace page (`https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/`).

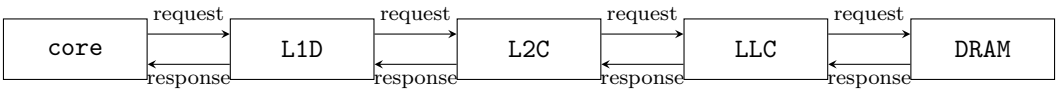
The cache hierarchy in ChampSim

Levels. The simulated core has two first-level caches, `L1I` for instructions and `L1D` for data. Below them is one second-level cache, `L2C`, and below that the last-level cache, `LLC`, which sits in front of main memory (`DRAM`). Each level is larger and slower than the one above it. These are the sizes in the given `champsim_config.json`, with 64-byte lines:

cache	sets	ways	size	hit latency (cycles)
`L1D`	64	12	48 KB	5
`L2C`	1024	8	512 KB	10
`LLC`	2048	16	2 MB	20

A miss in one level becomes a request to the level below, which ChampSim calls the *lower level*. The data comes back up the same path, and each level it passes through stores a copy.

Set and tag. A cache with S sets and W ways holds $S \times W$ lines. The low 6 bits of an address are the offset inside a 64-byte line. The next $\log_2 S$ bits pick the set, so the LLC uses 11 bits ($2^{11} = 2048$). The rest is the *tag*, which tells apart the lines that share a set. A line can go into any of the W ways of its set. When all W are full, one line must leave, and the *replacement policy* chooses which.



Requests and responses. Levels talk to each other with messages, which ChampSim calls packets (`PACKET` in `inc/block.h`). A request must carry everything the lower level needs to find the line and to know whom to answer:

`address`	the physical address; it gives the set and the tag
`type`	`LOAD`, `RFO`, `PREFETCH`, `WRITEBACK` or `TRANSLATION`
`ip`	the address of the instruction that made the access (its program counter)
`cpu`, `instr_id`	which core and which instruction is waiting for the data
`fill_level`	which levels keep a copy when the data comes back
`to_return`	the caches to send the response to

The response is the same packet sent back to every entry of `to_return`, now with the `data` field filled in. For the L1D the receiver is the core, for the L2C it is the L1D, for the LLC it is the L2C.

Queues and the MSHR. Each cache takes new packets from three queues and works through a fixed number of them per cycle (`max_read`, `max_write` in the JSON):

- `RQ`, the read queue: loads, and misses sent down by the level above.
- `WQ`, the write queue: stores at the L1D, writebacks of dirty lines at the lower levels.
- `PQ`, the prefetch queue. Prefetching is off in this lab, so it stays empty.

A miss takes tens to hundreds of cycles to come back, and the cache keeps serving other requests in the meantime. The *miss status holding register* (MSHR) remembers each miss that is still outstanding. One MSHR entry holds the packet of one missing line: its address, type, `to_return` list and the cycle it was sent. When a second request misses on a line that already has an entry, the cache adds the requester to that entry's `to_return` and sends nothing new down. When all entries are in use (`mshr_size`), a new miss waits in its queue until one frees up.

What happens to a read. In `src/cache.cc`, `handle_read` takes the oldest packet from the `RQ` and looks up its set and tag:

1. **Hit** (`readlike_hit`): copy the data into the packet, call `update_replacement_state` with `hit = 1`, and send the response to `to_return`.
2. **Miss** (`readlike_miss`): merge into an existing MSHR entry, or allocate a new one and put the request in the lower level's `RQ`.
3. **Fill** (`handle_fill`, later): when the lower level answers, pick a way in the set. An invalid way is used first; only when every way is valid is `find_victim` called. `filllike_miss` then writes the new line, calls `update_replacement_state` with `hit = 0`, and the response goes up to every entry of `to_return`. The MSHR entry is freed.

Stores, RFO and writebacks. Say a store is writing to cache line L. Even though store might modify only a few bytes of L, each cache line is stored as an entire unit in the cache and memory so if L is missing in the cache, we first need to "read" the line from below. This read is a *read for ownership* (RFO). When the line arrives it is written and marked `dirty`: it now differs from the copy in the level below. Clean lines can be dropped on eviction (nothing changed :D). A dirty line cannot be dropped it **must be written back**: `filllike_miss` sends it to the lower level's `WQ` as a `WRITEBACK` packet. In summary, the L1D receives stores from the core; the L2C receives `WRITEBACK`s from the L1D, and the LLC receives them from the L2C. *Note: With LRU at the L2C, a writeback almost always hits there (*399 959 *of* 400 091 *on* `soplex`*), because the line passed through the L2C on its way up and was used recently. A policy that evicts lines the L1D still holds makes writebacks miss.*

New concept: Bypassing. Some lines are used once and never again, for example a large array that is read from start to end. Storing such a line in the LLC evicts a line that might have been reused. A policy may therefore **bypass** the LLC and go straight to DRAM saving some precious LLC space. Implementation detail (not so important for now): `find_victim` returns `NUM_WAY` instead of a way, and the data goes up to the L2C without being stored. This ChampSim allows it only at the LLC, and never for a `WRITEBACK`, which must be stored somewhere. The policies in this lab do not bypass.

Policy logic and how it is used. The replacement policy can be broken into 3 subpolicies: **insertion, promotion and eviction**. In implementation in ChampSim there are two functions called from `cache.cc`: `update_replacement_state` is called on **every hit and every fill** (it's your entry point); and `find_victim` is called to determine which "way" to evict from a "set". There are 2 more functions — `initialize_replacement` is run once when champsim starts to let you initialize your data structures; and `replacement_final_stats` runs once at the end; ignore it.

Note on latency: The L1 caches answer in a few cycles, so their policy must be cheap to compute; LRU or something simpler. The L2C and the LLC have more time per access and see only the misses and writebacks of the levels above, so they can afford a policy with more state. In this lab you change only the L2C.

ChampSim

The folder `Champsim/` in the lab directory is a ready copy of ChampSim. You do not need to download anything during the lab.

To practise at home, clone the same version. Newer releases need extra libraries and change the code you write in this lab, so use the tag `2022-01`:

```
$ git clone --branch 2022-01 https://github.com/ChampSim/ChampSim.git Champsim
$ cd Champsim
```

The useful folders are:

`champsim_config.json`	the processor to build: cache sizes, latencies, policies
`config.sh`	reads the JSON and writes the `Makefile`
`replacement/`	one folder per cache replacement policy
`src/`, `inc/`	the simulator itself; only Task 1 changes it, through copies in `task1/`
`bin/`	the built simulators

Build and run ChampSim by hand (the canonical way)

Run these from inside `Champsim/`.

0. **Only if you built before:** `make clean`.
1. **Configure:** `./config.sh champsim_config.json`. It reads the JSON and writes the `Makefile`.
2. **Compile:** `make`. It builds the simulator into `bin/`, with the name given by `executable_name` in the JSON.
3. **Run:** give the binary a trace and two instruction counts.

```
$ ./config.sh champsim_config.json
$ make
$ ./bin/champsim --warmup_instructions 1000000 \
                 --simulation_instructions 10000000 \
                 /opt/CS231_Traces/soplex_66B.trace.xz
```

Changed the JSON or added a policy? Repeat steps 1 and 2. `make` alone does not read the JSON.

`config.sh` may print a `SyntaxWarning` about `\*`. Ignore it. The run options:

- `--warmup_instructions`: run these first to fill the caches. They are not counted.
- `--simulation_instructions`: run these next. They are counted.
- `--hide_heartbeat`: do not print a progress line every 10^7 instructions.
- The trace must end in `.xz` or `.gz`. Pass it as it is; do not decompress it.

Use two run lengths. A **quick run** (1M warm-up + 10M simulated) takes under a minute: use it to see that your code builds and behaves. A **long run** (20M + 25M) takes a few minutes per trace and gives the numbers worth comparing. The warm-up exists because a run starts with empty caches: the first misses only fill them, and counting them would make every policy look worse than it is.

Read the result. Everything after `Region of Interest Statistics` counts only the simulated instructions, not the warm-up. These are the lines we use:

```
CPU 0 cumulative IPC: [ipc] instructions: [n] cycles: [n]
cpu0_L1D TOTAL     ACCESS: [n]  HIT: [n]  MISS: [n]
cpu0_L2C TOTAL     ACCESS: [n]  HIT: [n]  MISS: [n]
LLC TOTAL          ACCESS: [n]  HIT: [n]  MISS: [n]
```

The two numbers that matter: IPC and L2C MPKI

- **IPC** (instructions per cycle; higher is better). Read it from the line `CPU 0 cumulative IPC: [ipc]`.
- **L2C MPKI** (L2C misses per 1000 instructions; lower is better). ChampSim does not print it. Compute it from the `MISS` of the `cpu0_L2C TOTAL` line and the `--simulation_instructions` you passed:

$$\text{L2C MPKI} = \frac{\text{L2C TOTAL MISS}}{\text{simulation instructions}/1000}$$

For example, 356 121 misses in 10 000 000 instructions give an L2C MPKI of 35.6121.

The lab makefile prints both for you, and `make check` compares exactly these two numbers with ours. MPKI works the same way for any cache: use that cache's `TOTAL` line.

Each cache also prints one row per request type below its `TOTAL` row: `LOAD`, `RFO` (a store), `PREFETCH`, `WRITEBACK` and `TRANSLATION`. `TRANSLATION` is the page-table reads of the address translation, which also pass through the L1D. **The L1D `TOTAL` row therefore counts more than the loads and stores of the program.** Say which row you use when you quote a miss count.

To see only these lines, pipe the output through `grep`:

```
$ ./bin/champsim [options] [trace].xz > output.txt
$ cat output.txt | grep -E 'cumulative IPC|TOTAL'
```

Choosing a replacement policy

When a cache set is full and a new line arrives, the *replacement policy* picks the line to evict. In ChampSim each cache has its own policy, set in `champsim_config.json`. The caches are `L1I`, `L1D`, `L2C` and `LLC`. The value you write in `"replacement": ...` corresponds to the name of a folder under `replacement/`. ChampSim provides some by default: `lru`, `srrip`, `drrip` and `ship`.

Every cache uses `lru` unless the JSON says otherwise. In the given JSON only the `LLC` block lists a policy. To change the policy of the L1D, add the line to its block, after a comma on the line before:

```
"L1D": {
    "sets": 64,
    "ways": 12,
    ...
    "prefetcher": "no",
    "replacement": "srrip"  <-- here
},
```

IMPORTANT NOTE: Also change `executable_name` at the top of the file as well, for example to `bin/l1d-srrip`, then run `./config.sh champsim_config.json && make`. This way you can keep making new binaries.

Writing your own policy. Make a folder `replacement/[name]/` with a file `[name].cc`, and set `"replacement": "[name]"` in the JSON. The file defines these four functions of `CACHE`. The names and arguments must be exactly these:

```
void CACHE::initialize_replacement()

uint32_t CACHE::find_victim(uint32_t cpu, uint64_t instr_id,
    uint32_t set, const BLOCK* current_set, uint64_t ip,
    uint64_t full_addr, uint32_t type)

void CACHE::update_replacement_state(uint32_t cpu, uint32_t set,
    uint32_t way, uint64_t full_addr, uint64_t ip,
    uint64_t victim_addr, uint32_t type, uint8_t hit)

void CACHE::replacement_final_stats()
```

- `initialize_replacement`: called once before the simulation. Set up your data here.
- `find_victim`: called when a line must be evicted from the full set `set`. Return the way, from 0 to `NUM_WAY - 1`, of the line to evict (`NUM_WAY` bypasses, see above). It is only called when all ways of the set are full.
- `update_replacement_state`: called when the line in `(set, way)` is hit (`hit` is 1) and when a new line is filled into it (`hit` is 0). It returns nothing, its just a trigger for you to do your policy changes.
- `replacement_final_stats`: called once at the end. It may print.

Inside these functions, the members of the cache are available. `NUM_SET` and `NUM_WAY` are parameters that define its shape, and `block[set * NUM_WAY + way]` defines one whole line. A line has the fields `valid`, `dirty`, `address`, `ip` (the program counter of the instruction that brought it in) and `lru`, an integer ChampSim reserves for the replacement policy. `type` is one of `LOAD`, `RFO`, `PREFETCH`, `WRITEBACK` and `TRANSLATION`.

Example: Random Policy. The simplest policy that builds and runs evicts a random way and keeps no state:

```
#include <cstdlib>

#include "cache.h"

void CACHE::initialize_replacement() {}

uint32_t CACHE::find_victim(uint32_t cpu, uint64_t instr_id,
    uint32_t set, const BLOCK* current_set, uint64_t ip,
    uint64_t full_addr, uint32_t type)
{
    // you can use standard C/C++ functions after the right #include
    return rand() % NUM_WAY;
}

void CACHE::update_replacement_state(uint32_t cpu, uint32_t set,
    uint32_t way, uint64_t full_addr, uint64_t ip,
    uint64_t victim_addr, uint32_t type, uint8_t hit)
{
}

void CACHE::replacement_final_stats() {}
```

Building and testing

You can do all of the above by hand (**please try it by hand!**), but the lab folder has a `makefile` that does it for you.

The lab makefile (run it from the lab folder, not from `Champsim/`)

`make task1`	build ChampSim with your `task1/` files; quick run on `soplex`
`make task2`	build your `task2/mylru.cc` at the L2C; quick run on `soplex`
`make task3`	the same for `task3/ip_based.cc`
`make lru`	ChampSim's own `lru` at the L2C: the baseline
`... trace=mcf`	use another trace: `soplex`, `bwaves`, `mcf` or `gcc`
`... long=1`	long run (20M + 25M) instead of quick
`make check`	check every task against our numbers
`make check-task2`	check one task (also `check-task1`, `check-task3`)
`make clean`	delete everything that was built

For Tasks 2 and 3, every run ends with the two numbers you need: the `CPU 0 cumulative IPC` line and the `cpu0_L2C MPKI` line.

`make task1` copies `Champsim/` to a hidden folder `.task1_champsim/`, puts your `task1/cache.h`, `cache.cc` and `main.cc` in place of the originals, builds it with the given JSON and runs it. The copy is made again on every build, so your edits never reach the other tasks.

For the other tasks, `make taskN` copies your file into `Champsim/replacement/[policy]/`, writes `Champsim/[policy].json` with that policy at the L2C and `executable_name` set to `bin/[policy]`, runs `config.sh` and `make`, and runs the binary. It prints the L2C rows and the *L2C MPKI*, which ChampSim does not print: the L2C `TOTAL` misses divided by the number of simulated instructions in thousands. The policy names are `mylru` (Task 2) and `ip_based` (Task 3). Edit only the file in the task folder; the copy under `Champsim/` is overwritten on every build.

`make check` runs each task for 1M + 5M instructions and compares the result with our solu-

tion's. Task 1 is compared exactly: the printed lines must equal `task1/expected_output.txt`. For the policy tasks, **the IPC and the L2C MPKI must both equal the values in `taskN/expected_output.txt` exactly**; if either differs, the check prints both your values and ours and reports FAIL. Each task checks only the traces where a wrong solution gives different numbers. ChampSim is deterministic, so a correct policy gives exactly our numbers.

With a correct `task2/mylru.cc` every trace passes. With the unchanged template, each trace shows both of your values and ours:

```
$ make check-task2
Building Champsim/bin/mylru (L2C replacement = mylru)
[task2] soplex  | FAIL (differs from the reference)
    got:      IPC 0.342978, L2C MPKI 48.1598
    expected: IPC 0.334436, L2C MPKI 34.0892
[task2] mcf     | FAIL (differs from the reference)
    got:      IPC 0.627381, L2C MPKI 23.3422
    expected: IPC 0.647025, L2C MPKI 14.1932
...
```

The traces run at the same time, so the lines can come in any order. `make check` also accepts a single task: `make check-task1`, `make check-task2` or `make check-task3`.

If a test keeps failing for no reason you can see, run `make clean`.

Structure of submission/templates

Edit only the files marked `(TODO)`.

Submission format:

```
your-roll-no/
|- task1/
    |- cache.h (TODO)
    |- cache.cc (TODO)
    |- main.cc (TODO)
    |- expected_output.txt
|- task2/
    |- mylru.cc (TODO)
    |- expected_output.txt
|- task3/
    |- ip_based.cc (TODO)
    |- expected_output.txt
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz` in lowercase. The command given below can be used to do the same

```
$ ./build_archive.sh
Enter Student Roll Number: your-roll-no
Packaging assignment files...
  [+] Copied: ...
Successfully created your-roll-no.tar.gz
```

Tasks

Task 1: Counting Events in the L2 Cache

ChampSim prints how many hits and misses each cache had, but not where in the cache they happened. In this task you add a counter to the L2C, and optionally a second one, and print them with the other statistics. To place a counter you must find the line of `src/cache.cc` where the event happens, so the task is mainly a reading exercise on the code described in *The cache hierarchy in ChampSim*.

- **Read hits per way**: for each of the 8 ways, the number of read hits in that way.
- **Most misses in one set** (**optional**, do it after the rest of the lab): for each of the 1024 sets, the number of misses in that set; print the largest count and its set.

Step 1: Storage. Every cache is one object of the class `CACHE` in `inc/cache.h`. Declare both counters as members of the class, so that each cache gets its own. Use `std::vector<uint64_t>`: one entry per way (`NUM_WAY` entries) and one per set (`NUM_SET` entries), all 0. A member can be given its size where it is declared:

```
std::vector<uint64_t> name = std::vector<uint64_t>(NUM_WAY);
```

This works because `NUM_WAY` is declared earlier in the class, and members are set up in the order they are declared.

Step 2: Where a read hit happens. `handle_read` takes a packet from the read queue and calls `get_way`. A result smaller than `NUM_WAY` is a hit, and the result is the way that hit. Count it there, before `readlike_hit` is called.

Step 3: Where a miss is recorded. ChampSim counts a miss in `filllike_miss`, which writes the new line into its set: read misses reach it through `handle_fill` when the data comes back, write misses through `handle_writeback`. It adds 1 to `sim_miss`, the counter behind the `MISS` column. A second miss to a line that already has an MSHR entry is merged and never reaches `filllike_miss`, so it is not counted there either. Its argument `set` is the set of the line.

Step 4: Skip the warm-up. The printed statistics cover only the instructions after the warm-up. Your counters must do the same: count only when `warmup_complete[handle_pkt.cpu]` is true. The `DP(...)` debug blocks at the top of `readlike_hit` and `filllike_miss` use the same check.

Step 5: Printing. At the end of the run, `main` calls `print_roi_stats(cpu, cache)` in `src/main.cc` once for each cache; it prints the `TOTAL`, `LOAD`, ... rows. At the end of that function, for the L2C only (`cache->NAME == "cpu0_L2C"`), print exactly these two lines (only the first if you skip the optional counter):

```
cpu0_L2C READ HITS PER WAY: h0 h1 h2 h3 h4 h5 h6 h7
cpu0_L2C MOST MISSES IN ONE SET: m (set s)
```

`h0` ... `h7` are the read hits of ways 0 to 7, separated by single spaces. `m` is the largest miss count of any set and `s` is that set; if several sets have `m` misses, `s` is the lowest of them.

The first counter is required. The second is optional: `make check` reports it separately, and skipping it does not fail Task 1. If you skip it, leave out Steps 3 and 4 for the miss counter and print only the first line.

Task. Edit `task1/cache.h`, `task1/cache.cc` and `task1/main.cc`. They are copies of `Champsim/inc/cache.h`, `Champsim/src/cache.cc` and `Champsim/src/main.cc`; the `TODO` comments in `cache.h`, `cache.cc` and `main.cc` mark where the declarations, the counting and the printing go. **Change nothing else in these files: the other statistics, and therefore the grading, depend on them.** Edit the copies in `task1/`, not the files under `Champsim/`.

With the solution, the quick run on `soplex` prints:

```
$ make task1
Building .task1_champsim/bin/champsim with task1/cache.h, cache.cc, main.cc
.task1_champsim/bin/champsim --warmup_instructions 1000000 \
    --simulation_instructions 10000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
cpu0_L2C TOTAL     ACCESS:     630218  HIT:     274097  MISS:     356121
cpu0_L2C LOAD      ACCESS:     380439  HIT:      74768  MISS:     305671
cpu0_L2C READ HITS PER WAY: 10614 10593 10394 10736 10535 10568 10652 10531
cpu0_L2C MOST MISSES IN ONE SET: 506 (set 415)
```

Task 2: Least Recently Used Replacement

LRU evicts the line of the set that was used longest ago. It rests on the observation that a line used recently is likely to be used again soon. ChampSim ships an LRU (`replacement/lru/lru.cc`), but it is written with C++ iterators and lambdas around the `lru` field of `BLOCK`. In this task you write the same policy with a plain array and loops, and show that it is the same policy: the simulator must give exactly the same result with yours as with ChampSim's.

Step 1: An age per line. Give every line of a set an age from 0 to $W - 1$, where W is `NUM_WAY`. Each age is used by exactly one line of the set. $W - 1$ is the most recently used line, 0 the least recently used.

- **Eviction** (`find_victim`): return the way whose age is 0.
- **Hit and fill** (`update_replacement_state`): let a be the age of the line in `(set, way)`. Every line of the set whose age is greater than a loses 1. Then the line in `(set, way)` gets age $W - 1$.

The second rule keeps the ages a permutation of $0 \dots W - 1$: the lines above a move down by one and fill the gap left by a, and $W - 1$ becomes free for the line just used.

Step 2: A worked example. Take one set with $W = 4$ ways holding lines A, B, C and D. Each row shows the ages after the event on the left.

event	A	B	C	D	
start	0	1	2	3	D used last, A longest ago
hit on B	0	3	1	2	C and D were above 1, they lose 1
hit on D	0	2	1	3	only B was above 2
miss, set full					`find_victim` returns way 0 (A)
fill E into way 0	3	1	0	2	E was 0, so everyone loses 1

Step 3: The starting state. All ages start at 0, so before the set is full they are not a permutation. They need not be. While a set has an invalid way, `cache.cc` fills that way without calling `find_victim`. Each fill makes the new line $W - 1$ and moves the other filled lines down by one. After W fills the ages are $0 \dots W - 1$ again, in fill order.

Step 4: Storage. The ages live in a `std::vector<uint32_t>` of size `NUM_SET` × `NUM_WAY`; the age of `(set, way)` is at index `set * NUM_WAY + way`. The template declares it as `lruBits[this]`, a map from the cache to its vector, so that two caches using the same policy do not share their ages.

Step 5: Writebacks. A writeback that hits is the level above returning a dirty line. The program did not use the line, so it must not change its age. The template returns early for this case, as ChampSim's `lru` does.

Task. Fill in `initialize_replacement`, `find_victim` and `update_replacement_state` in `task2/mylru.cc`. Do not read or write the `lru` field of `BLOCK`; keep the ages in `lruBits`.

Reference numbers: IPC and L2C MPKI

Long runs (`make task2 long=1 trace=[trace]`) on the lab machines give these numbers. **Your IPC and L2C MPKI must match the `mylru` rows.** The `lru` rows are ChampSim's own LRU at the L2C, for comparison.

trace	L2C policy	IPC	L2C MPKI	L1D miss latency (cycles)
`soplex`	`lru`	0.323267	34.0741	310.9
	`mylru`	0.323267	34.0741	310.9
`bwaves`	`lru`	0.243644	26.1799	1400.8
	`mylru`	0.243644	26.1799	1400.8
`mcf`	`lru`	0.409243	16.9194	86.3
	`mylru`	0.409243	16.9194	86.3
`gcc`	`lru`	0.113813	69.9909	113.5
	`mylru`	0.113813	69.9909	113.5

Your policy must give the same IPC as ChampSim's `lru` on every trace, to the last digit. Then you know the two policies evict the same line every time.

On `soplex`, the long runs of both give the same numbers, to the last cycle:

```
$ make lru long=1
Building Champsim/bin/lru (L2C replacement = lru)
Champsim/bin/lru --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.323267 instructions: 25000002 cycles: 77335507
cpu0_L2C TOTAL     ACCESS:    1434603  HIT:     582750  MISS:     851853
cpu0_L2C LOAD      ACCESS:     884477  HIT:     161932  MISS:     722545
cpu0_L2C RFO       ACCESS:     149423  HIT:      20421  MISS:     129002
cpu0_L2C WRITEBACK ACCESS:     400091  HIT:     399959  MISS:        132
cpu0_L2C MPKI: 34.0741

$ make task2 long=1
Building Champsim/bin/mylru (L2C replacement = mylru)
Champsim/bin/mylru --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.323267 instructions: 25000002 cycles: 77335507
cpu0_L2C TOTAL     ACCESS:    1434603  HIT:     582750  MISS:     851853
cpu0_L2C LOAD      ACCESS:     884477  HIT:     161932  MISS:     722545
cpu0_L2C RFO       ACCESS:     149423  HIT:      20421  MISS:     129002
cpu0_L2C WRITEBACK ACCESS:     400091  HIT:     399959  MISS:        132
cpu0_L2C MPKI: 34.0741
```

Task 3: Replacement by Instruction Address

Every line in a cache was brought in by one load or store instruction, and every instruction has an address, its `ip` (instruction pointer, also called program counter). Some instructions load data that is used again soon, such as a counter or a small table. Others load data that is used once, such as one element of a large array scanned from start to end. A policy that learns which instructions are which can keep the lines of the first kind and evict the lines of the second.

Step 1: A counter per instruction. Keep an 8-bit counter, `pcCount`, for each instruction. A high count means the lines that instruction brings in tend to be hit again.

- **Fill** (`hit = 0`): the instruction `ip` missed. Decrease its counter by 1, but not below 0.
- **Hit** (`hit = 1`): the instruction `ip` found its data. Increase its counter by 1, but not above 255.
- **Eviction** (`find_victim`): `current_set[i].ip` is the `ip` of the instruction that brought line `i` in. Evict the line whose `ip` has the lowest counter. On a tie, evict the line with the lowest way.

Hits and fills use the `ip` argument of `update_replacement_state`: the instruction making the access now. Eviction uses the `ip` stored in each line: the instruction that brought it in. Writeback hits are skipped, as in Task 2.

Step 2: Bounded storage. Addresses in a trace are 48 bits wide. A counter for every possible `ip` would need 2^{48} bytes, and hardware has a fixed table. Use only the lower 16 bits of the `ip` (`ip & 0xFFFF`) as the index into a table of $2^{16} = 65536$ counters. Two instructions whose addresses agree in the low 16 bits share a counter. The template gives you `pc_index(ip)`, which computes this index.

Step 3: A worked example. A set of $W = 4$ ways holds lines brought in by instructions with these counters:

way	`current_set[way].ip & 0xFFFF`	`pcCount`
0	`0x1a40`	7
1	`0x1a58`	0
2	`0x2210`	3
3	`0x1a58`	0

Ways 1 and 3 tie at 0, so `find_victim` returns 1. If instruction `0x1a58` now hits on a line, its counter becomes 1, and the next victim is still way 1 (count 1, lower than 3 and 7).

Task. Fill in `initialize_replacement`, `find_victim` and `update_replacement_state` in `task3/ip_based.cc`. The table is declared for you as `pcCount[this]`, a `std::vector<uint8_t>`.

Reference numbers: IPC and L2C MPKI

Long runs (`make task3 long=1 trace=[trace]`) on the lab machines give these numbers. **Your IPC and L2C MPKI must match the `ip_based` rows.** The `lru` rows are ChampSim's own LRU at the L2C, for comparison.

trace	L2C policy	IPC	L2C MPKI	L1D miss latency (cycles)
`soplex`	`lru`	0.323267	34.0741	310.9
	`ip_based`	0.333429	46.6156	287.9
`bwaves`	`lru`	0.243644	26.1799	1400.8
	`ip_based`	0.243644	34.9085	1400.9
`mcf`	`lru`	0.409243	16.9194	86.3
	`ip_based`	0.402809	25.0904	91.3
`gcc`	`lru`	0.113813	69.9909	113.5
	`ip_based`	0.114147	69.8671	113.1

```
$ make task3 long=1
Building Champsim/bin/ip_based (L2C replacement = ip_based)
Champsim/bin/ip_based --warmup_instructions 20000000 \
    --simulation_instructions 25000000 \
    /opt/CS231_Traces/soplex_66B.trace.xz
CPU 0 cumulative IPC: 0.333429 instructions: 25000002 cycles: 74978442
cpu0_L2C TOTAL     ACCESS:    1434756  HIT:     269365  MISS:    1165391
cpu0_L2C LOAD      ACCESS:     884487  HIT:      98827  MISS:     785660
cpu0_L2C RFO       ACCESS:     149488  HIT:      24935  MISS:     124553
cpu0_L2C WRITEBACK ACCESS:     400169  HIT:     145300  MISS:     254869
cpu0_L2C MPKI: 46.6156
```

Compare the `ip_based` rows with the `lru` rows in the table above: the same policy can help one program and hurt another.